

Tổng Hợp Kiến Thức

1. Khái niệm cốt lõi

- **Xác thực (Authentication):** Xác minh **bạn là ai**. Ví dụ: Đăng nhập bằng email và mật khẩu.
- **Phân quyền (Authorization):** Xác định **bạn được phép làm gì**. Ví dụ: Chỉ admin mới được xóa todo.
- **Trọng tâm:** Hướng dẫn này tập trung vào **Xác thực** (sử dụng access token). Phân quyền thường được xử lý bởi backend.

2. Luồng đăng nhập

2.1 Trang đăng nhập (`src/app/login/page.jsx`)

- **Mục đích:** Tạo trang đăng nhập để xác thực người dùng và lưu access token vào cookie.
- **Tính năng chính:**
 - Sử dụng React `useState` để quản lý input (email, mật khẩu) và xử lý lỗi.
 - Gọi API backend (`/api/auth/login`) để xác thực.
 - Chuyển hướng đến dashboard khi thành công bằng `useRouter` từ Next.js.
 - Hiển thị thông báo lỗi khi đăng nhập thất bại.

```
'use client';
import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const router = useRouter();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    try {
      const res = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
      });
      if (res.ok) {
        router.push('/dashboard');
      } else {
        const data = await res.json();
        setError(data.message || 'Email hoặc mật khẩu không đúng.');
```

```

    value={email}
    onChange={(e) => setEmail(e.target.value)}
    placeholder="Email"
    required
  />
  <br />
  <input
    type="password"
    value={password}
    onChange={(e) => setPassword(e.target.value)}
    placeholder="Mật khẩu"
    required
  />
  <br />
  <button type="submit">Đăng nhập</button>
  {error && <p style={{ color: 'red' }}>{error}</p>}
</form>
</div>
);
}

```

2.2 API Xác thực (`src/app/api/auth/login/route.js`)

- **Mục đích:** Mô phỏng API xác thực (thực tế nên dùng backend framework như Express.js, Nest.js,...).
- **Tính năng chính:**
 - Kiểm tra email và mật khẩu (mô phỏng, thực tế cần truy vấn database và kiểm tra mật khẩu đã hash).
 - Tạo và lưu access token vào cookie với các thuộc tính bảo mật.

```

import { NextResponse } from 'next/server';
import { cookies } from 'next/headers';

export async function POST(request) {
  const body = await request.json();
  const { email, password } = body;

  // Mô phỏng xác thực
  if (email !== 'admin@letdiv.com' || password !== '123') {
    return NextResponse.json(
      { message: 'Email hoặc mật khẩu không đúng' },
      { status: 401 }
    );
  }

  // Tạo và set cookie
  const accessToken = 'fake-jwt-token-string-for-example';
  const cookieStore = await cookies();
  cookieStore.set('access_token', accessToken, {
    httpOnly: true, // Ngăn JS client truy cập
    secure: process.env.NODE_ENV === 'production', // Chỉ dùng HTTPS ở production
    maxAge: 60 * 60 * 24 * 7, // 1 tuần
    path: '/', // Áp dụng toàn website
  });

  return NextResponse.json({ message: 'Đăng nhập thành công' });
}

```

2.3 Trang Dashboard (`src/app/dashboard/page.jsx`)

- **Mục đích:** Hiển thị trang dashboard sau khi đăng nhập thành công.

```
export default function DashboardPage() {
  return (
    <div>
      <h1>Chào mừng đến với Dashboard!</h1>
      <p>Bạn đã đăng nhập thành công.</p>
    </div>
  );
}
```

- **Lưu ý:**
 - Sử dụng Developer Tool để kiểm tra cookie sau khi đăng nhập.
 - Giải thích cơ chế cookie nếu cần (httpOnly, secure, maxAge, path).

3. Middleware

3.1 Middleware là gì?

- **Định nghĩa:** Đoạn code chạy **trước** khi request được xử lý, đứng giữa request và response.
- **Mục đích xác thực:** Kiểm tra xem người dùng có token hợp lệ để truy cập route được bảo vệ hay không.

3.2 Tạo Middleware (`src/middleware.js`)

- **Mục đích:** Bảo vệ các route như `/dashboard` bằng cách kiểm tra access token.
- **Tính năng chính:**
 - Kiểm tra `access_token` trong cookie.
 - Chuyển hướng về trang đăng nhập nếu không có token.
 - Sử dụng `matcher` để chỉ áp dụng Middleware cho các route cụ thể.

```
import { NextResponse } from 'next/server';

export function middleware(request) {
  const token = request.cookies.get('access_token')?.value;

  if (!token) {
    const loginUrl = new URL('/login', request.url);
    return NextResponse.redirect(loginUrl);
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*'], // Áp dụng cho /dashboard và các route con
};
```

- **Giải thích `matcher` :**
 - `matcher: ['/dashboard/:path*']` : Áp dụng Middleware cho:
 - `/dashboard`
 - `/dashboard/settings`
 - `/dashboard/users/profile`
 - `/dashboard/orders/123`
 - `:path*` khớp với mọi đường dẫn con của `/dashboard` .

4. Lưu ý

- **Bảo mật cookie:**
 - `httpOnly: true` : Ngăn JavaScript truy cập cookie.
 - `secure: true` (ở production): Chỉ gửi cookie qua HTTPS.
 - `maxAge` : Thời gian sống của cookie (ví dụ: 1 tuần).
 - `path: '/'` : Áp dụng cho toàn bộ website.